

Final Report

Operating System Project

Department of Software Engineering

Remote Command-Driven Windows Application Launcher

AASIM NAZIM

(BSE-23F-146)

HADIYA MUNIR

(BSE-23F-147)

AHMED SHAHI

(BSE-23F-110)

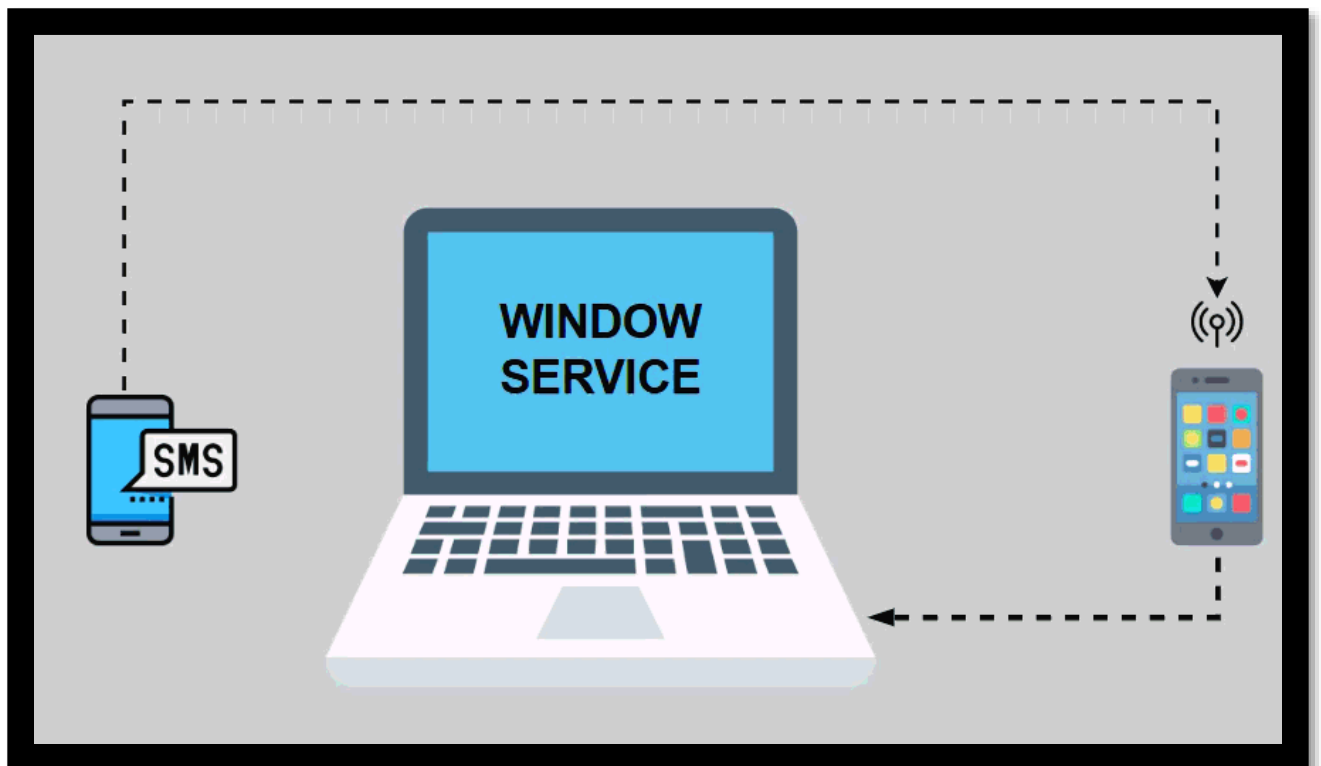
Assigned By: SIR ASIF IRSHAD

1. Motivation

The motivation for developing this project emerged after carefully analyzing various project ideas that could practically demonstrate core operating system and software engineering concepts. The increasing need for remote system control without relying on complex graphical interfaces or continuous internet connectivity led to the selection of this project. Significant effort was invested in choosing a problem that is technically challenging, academically relevant, and practically useful. This project was specifically chosen to explore how Windows services, inter-process communication, and mobile-based command inputs can be combined to automate desktop operations efficiently. The motivation also includes gaining hands-on experience with background services, process management, and secure communication mechanisms within an operating system environment

2. Overview

The core idea of this project is to enable remote control of desktop system operations using simple SMS-based commands. The system allows a user to control applications such as Microsoft Word and PowerPoint and perform system shutdown operations without direct physical access to the computer. The project integrates mobile communication, background services, and inter-process communication to provide a lightweight and reliable automation solution.



2.1 Significance / Importance of the Project

The importance of this project lies in its practical application and academic value. From a practical perspective, it provides a simple and effective solution for remotely controlling a desktop system in situations where traditional remote access tools may not be available or suitable. The project demonstrates how automation can be achieved using basic technologies such as SMS and local inter-process communication.

Academically, the project covers complex and important areas including Windows service architecture, process monitoring, inter-process communication using Named Pipes, and secure execution of system-level commands. The difficulty of working with non-interactive service sessions and ensuring safe execution of user-interface applications adds to the technical depth of the project. If successfully implemented, this system can serve as a foundation for more advanced automation and remote control solutions.

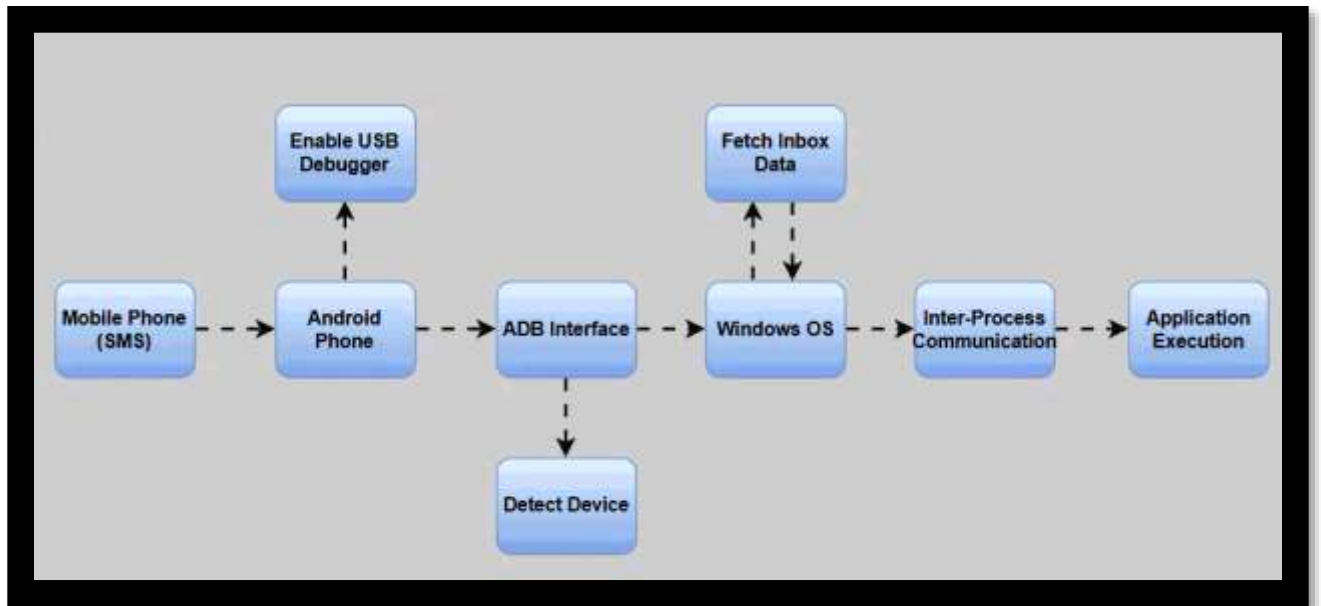
2.2 Description of the Project

This project focuses on solving the problem of remotely controlling desktop applications and system operations using mobile phone SMS commands. The system consists of two main components: a Windows Service and a Desktop Agent. The Windows Service runs continuously in the background, and communication between the Windows Service and the Desktop Agent is handled using a secure Named Pipe-based mechanism.

An Android mobile phone is connected to the system, and ADB (Android Debug Bridge) is used to access and read the SMS inbox of the device. Incoming SMS messages are processed, filtered, and validated before being converted into executable commands. This approach enables reliable command retrieval without requiring internet connectivity.

The Desktop Agent operates within the interactive user session and executes commands such as opening Microsoft Word, opening Microsoft PowerPoint, closing applications, and shutting down the system. The project follows a specific-purpose approach rather than a generalized one, as it is designed primarily for Windows-based systems, selected desktop applications, and Android devices connected via ADB.

SYSTEM ARCHITECTURE:



Problems Encountered and Solutions

1. Word Opening in Background

- **Problem:** Word opened in the background when launched from the service.
- **Solution:** Application execution was shifted to a desktop agent running in the interactive user session.

2. Windows Service Cannot Show UI Applications

- **Problem:** Windows services run in Session 0 and cannot display UI apps.
- **Solution:** A separate desktop agent was used to launch applications, with communication via Named Pipes.

3. Shutdown Executed While Apps Were Running

- **Problem:** Shutdown occurred even when Word or PowerPoint was open.
- **Solution:** A 10-second delay was added after detecting running applications. Shutdown execute when all running applications will close after 10 seconds.

4. Multiple Commands in One Message

- **Problem:** Messages containing multiple commands were not handled properly.
- **Solution:** Commands were parsed and executed sequentially.

5. ADB SMS Filtering Issue

- **Problem:** ADB returned the complete SMS inbox along with metadata, making it difficult to identify new and valid commands.
- **Solution:** A time-based filtering logic was applied by comparing SMS timestamps with the system's current time. Messages with a time difference of ≤ 7 ms were treated as new commands, while older messages were ignored.

2.3 Related Work and Literature Review

The development of this project is based on existing research and tools related to system automation and remote command execution. Background study includes Windows Service development using the pywin32 library, inter-process communication through Named Pipes, and process monitoring using psutil. The project also draws inspiration from prior work on SMS-based automation systems and remote device control techniques. Official documentation of Python, Microsoft Windows Services, and Android Debug Bridge (ADB) was reviewed to understand reliable implementation methods. These resources provided a strong foundation for designing a secure and efficient command execution framework.

3. Salient Features

3.1 SMS-Based Desktop Control via Android (ADB)

This project allows remote control of desktop applications using SMS commands sent from an Android phone. The system reads the SMS inbox directly from the connected Android device using ADB, eliminating the need for internet-based messaging services.

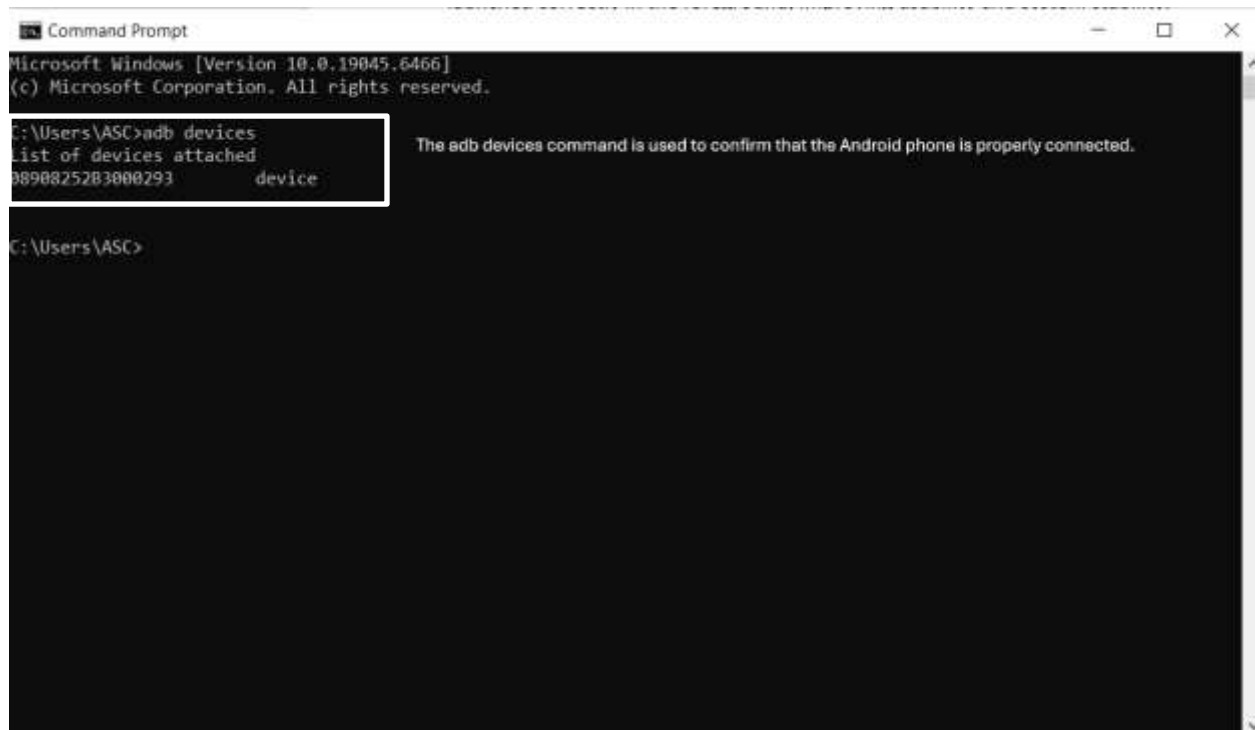
Library Used	Purpose of Library	Usage in Project
subprocess	Executes ADB commands	Used to read Android SMS inbox and run commands
re	Extracts SMS content and metadata	Identifies valid command messages from SMS
psutil	Process monitoring	Checks if Word/PPT are running

```
command = 'adb shell "content query --uri content://sms/inbox"'

result = subprocess.run(
    command,
    stdout=subprocess.PIPE,
    stderr=subprocess.PIPE,
    shell=True
)
```

```
for row in rows:
    date_match = re.search(r"date=(\d+)", row)
    body_match = re.search(r"body=(.*?)(?=\s, \w+=)", row, re.DOTALL)
```

ADB Verification:



The screenshot shows a Windows Command Prompt window titled "Command Prompt". The window displays the following text:

```
Microsoft Windows [Version 10.0.19045.6466]
(c) Microsoft Corporation. All rights reserved.

C:\Users\ASC>adb devices
List of devices attached
3890825283000293    device
```

To the right of the command prompt output, there is a note: "The adb devices command is used to confirm that the Android phone is properly connected."

Supported Commands:

Command	Action Performed
open word	Opens Microsoft Word on the desktop
open ppt	Opens Microsoft PowerPoint on the desktop
shutdown	Safely shuts down the system
open word, open ppt, shutdown	Executes multiple commands sequentially
closed word	Closes Microsoft Word if running
closed ppt	Closes Microsoft PowerPoint if running

3.2 SMS Recency Filtering Mechanism (Metadata-Based Validation)

ADB returns the complete SMS inbox including old messages and metadata. To avoid executing outdated commands, a time-based filtering mechanism is implemented to detect only newly received and valid SMS commands.

Library Used	Purpose of Library	Usage in Project
time	Compares SMS timestamp with current system time	Prevents execution of old or duplicate SMS commands by allowing only recent messages (≤ 7 seconds)

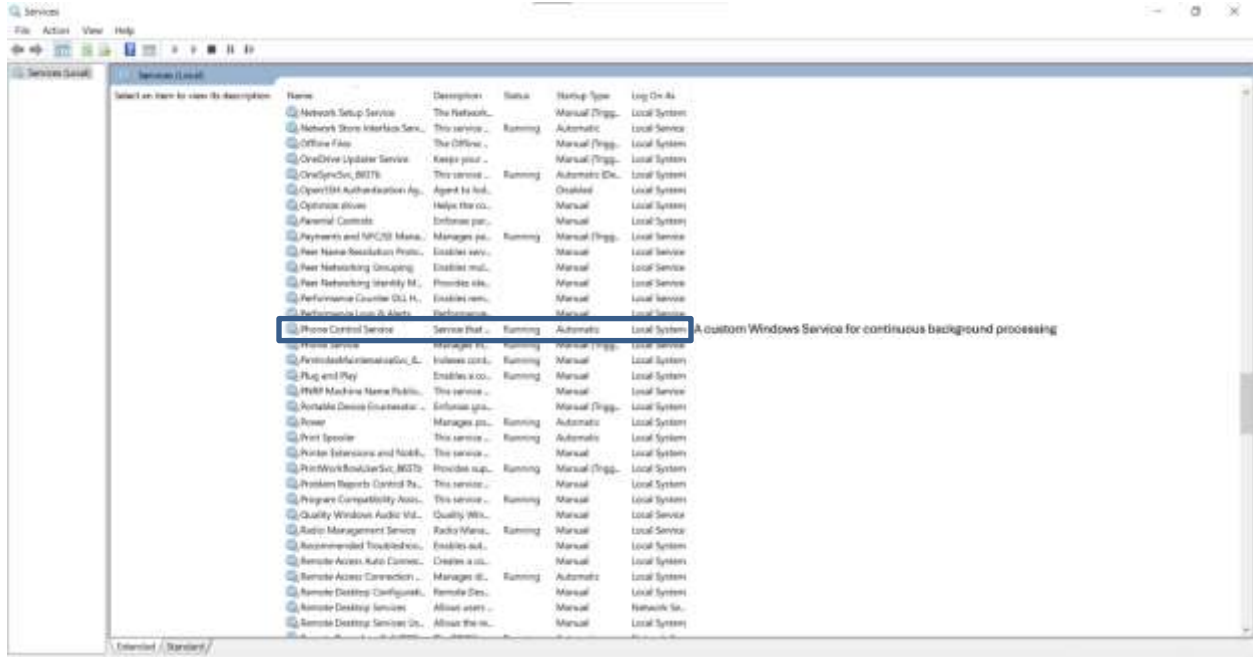
```
current_time_ms = int(time.time() * 1000)
time_diff = current_time_ms - sms_date
if time_diff <= 7000:
    return True
```

3.3 Windows Service for Background Execution

A Windows Service is implemented to ensure the system runs continuously in the background, even when no user is logged in.

Library Used	Purpose of Library	Usage in Project
pywin32 (win32serviceutil, win32service, win32event)	Provides Windows Service APIs	Allows the system to run persistently in the background without user interaction

```
class PhoneControlService(win32serviceutil.ServiceFramework):
    _svc_name_ = "PhoneControlService"
    _svc_display_name_ = "Phone Control Service"
```



3.4 Secure Inter-Process Communication Using Named Pipes

The project uses Named Pipes for secure and fast communication between the Windows Service and Desktop Agent.

Library Used	Purpose of Library	Usage in Project
win32pipe	Implements Named Pipe communication	Enables secure data exchange between service and agent
win32file	Supports reading/writing through pipes	Facilitates communication with controlled access

```
pipe = win32pipe.CreateNamedPipe(
    PIPE_NAME,
    win32pipe.PIPE_ACCESS_DUPLEX,
    win32pipe.PIPE_TYPE_MESSAGE |
    win32pipe.PIPE_READMODE_MESSAGE |
    win32pipe.PIPE_WAIT,
    1, 65536, 65536, 0, None
)
```

3.5 Desktop Agent for Interactive Application Execution

Since Windows services cannot launch UI applications, a desktop agent runs in the interactive user session to execute commands like opening Word or PowerPoint.

Library Used	Purpose of Library	Usage in Project
subprocess	Launches applications	Ensures Microsoft Word and PowerPoint are opened in the user's interactive session
threading	Allows parallel execution	Enables opening multiple applications simultaneously

```
process = subprocess.Popen([WORD_PATH])
```

```
if __name__ == "__main__":  
    threading.Thread(target=sms_poller, daemon=True).start()  
    run_pipe_server()
```

3.6 Safe Shutdown with Application Monitoring

Before executing sensitive operations like shutdown, the system checks if Microsoft Word or PowerPoint is currently running and system displays a warning popup and waits before shutting down if Office applications are running, ensuring data safety.

Library Used	Purpose of Library	Usage in Project
psutil	Detects running processes	Ensures Word and PowerPoint are closed before shutdown
ctypes	Provides Windows API access	Shows shutdown warning popup
os	Executes system commands	Performs system shutdown safely

```
for proc in psutil.process_iter(['pid', 'name', 'status']):  
    try:  
        if proc.status() != psutil.STATUS_RUNNING:
```

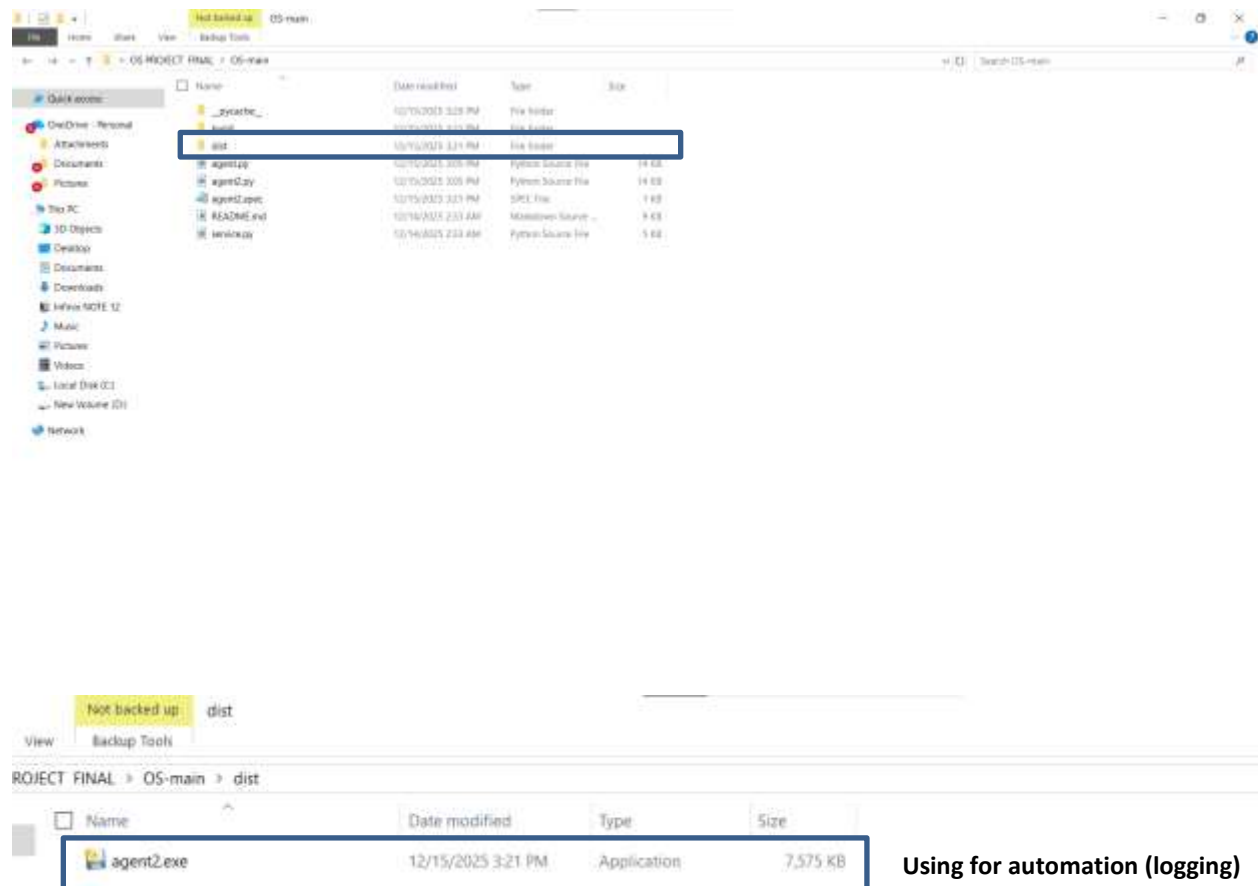
```
# Windows shutdown command  
os.system("shutdown /s /t 1")
```

3.7 Automated Agent Execution

The project enables the agent to run automatically in the background, execute commands, and track activities without manual intervention.

Library Used	Purpose of Library	Usage in Project
logging	Records execution flow and system events	Enables automated execution of the agent with proper activity tracking and debugging support

```
process = subprocess.Popen([WORD_PATH])
executing_processes["open word"] = process
logging.info("🔥 Command executed: open_word")
```



4. Project Planning

This section describes the project planning and task distribution for successful project completion within the given timeline.

4.1 Team Member Responsibilities

Member 1 (ADB & SMS Handling)

Responsible for configuring ADB and handling SMS reading and command extraction from the Android device.

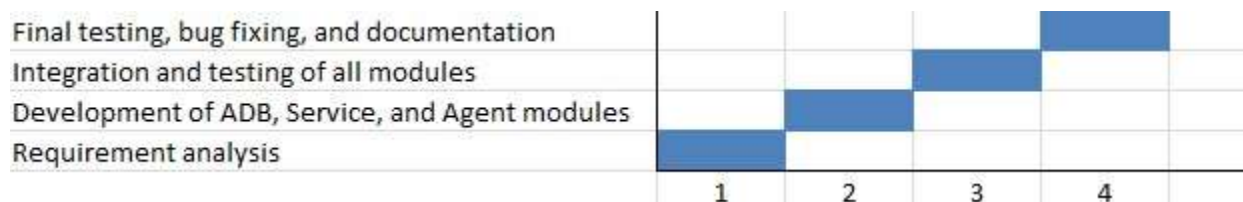
Member 2 (Windows Service Development)

Responsible for developing the Windows Service, managing background execution, and forwarding commands to the desktop agent using Named Pipes.

Member 3 (Desktop Agent Development)

Responsible for developing the desktop agent to execute commands such as opening applications and performing system shutdown.

4.2 Gantt Chart:



5. Complete System Configuration and Setup

This section describes the complete configuration required to deploy, execute, and maintain the Remote Command-Driven Windows Application Launcher. The configuration ensures correct system behavior, security, and reproducibility of the project.

5.1 Hardware Configuration

- A Windows-based PC or Laptop
- Android Smartphone (any modern Android version)
- USB cable for connecting the Android device to the PC

5.2 Software Configuration

- **Operating System:** Windows 10 or Windows 11
- **Programming Language:** Python 3.x
- **Microsoft Applications:** Microsoft Word, Microsoft PowerPoint

- **Android Tools:** Android Debug Bridge (ADB) installed and configured
- Python Libraries and Modules

5.3 Android Device Configuration

To enable SMS-based command detection, the Android device must be configured as follows:

- Developer Mode enabled
- USB Debugging enabled
- Device connected to PC via USB
- Device verification confirmed using the command: **adb devices**

This ensures that only authorized and physically connected devices can send commands.

5.4 Windows Service Configuration

A custom Windows Service is used for continuous background processing:

- The service starts automatically with the operating system
- Runs without any user interface
- Ensures 24/7 availability of the system
- Installed using Python service installation commands
- Window service act as a client that runs as along term process ensures agent connectivity.

Due to Windows Session 0 isolation, the service does not interact directly with the graphical user interface.

5.5 Agent Configuration

The system includes a desktop agent (**agent2.exe**) responsible for executing GUI-based commands:

- Runs in the user session
- Executes application launch and shutdown commands
- Deployed inside the **dist directory**
- Operates completely in the background without GUI
- Agent act as a server which create named pipe and run pipe server

5.6 Inter-Process Communication Configuration

The project uses Named Pipes for secure inter-process communication between the Windows Service and the Agent.

Pipe Name: [\\.\pipe\phone_control_pipe](#)

- Message-based communication is used
- Ensures reliable and secure local communication
- Enables separation between background service logic and GUI execution

5.7 Application Path Configuration

Absolute paths are configured for Microsoft Office applications to ensure reliable execution:

- Microsoft Word executable path
- Microsoft PowerPoint executable path

This avoids dependency on system environment variables.

5.8. SMS Processing Configuration

- SMS inbox is read using ADB content provider
- All SMS messages are retrieved and filtered
- Only the latest SMS is processed
- Commands are executed only if SMS timestamp is within a predefined threshold
- Old or repeated SMS commands are ignored

This mechanism prevents duplicate execution and enhances security.

5.9 Logging Configuration

Since the agent runs as a background executable without a graphical interface, logging is enabled:

- Logs are written to a log file in the execution directory
- Records system events, command execution, warnings, and errors
- Provides execution history and debugging support
- Acts as the primary monitoring mechanism for the system

5.10 Security Configuration

The following security measures are implemented:

- Physical USB connection required
- USB debugging enabled manually by user
- SMS commands executed only if recent
- Limited and predefined command set
- No internet connectivity required

These measures ensure controlled and authorized system access.

6. Libraries / Modules / APIs Used in the Project

Library / Module	Version	Purpose	Reference
Python	3.x	Core programming language used to develop the Windows Service and Desktop Agent	Official Python Documentation
pywin32	Latest stable	Provides access to Windows APIs for creating Windows Services, Named Pipes, and system-level operations	pywin32 Documentation
win32serviceutil	Included in pywin32	Used to create, install, start, and manage the Windows Service	Microsoft Windows API
win32pipe	Included in pywin32	Implements Named Pipe communication between Service and Desktop Agent	Microsoft Windows API
win32file	Included in pywin32	Handles file and pipe read/write operations	Microsoft Windows API
pywintypes	Included in pywin32	Supports Windows-specific data types and error handling	pywin32 Documentation
psutil	Latest stable	Used to monitor running processes and check application status	psutil Documentation
subprocess	Built-in	Executes external programs such as Office applications and ADB commands	Python Standard Library
threading	Built-in	Enables parallel execution of SMS polling and pipe communication	Python Standard Library
os	Built-in	Performs system-level operations such as shutdown commands	Python Standard Library
time	Built-in	Used for delays, polling intervals, and timestamp handling	Python Standard Library
re	Built-in	Used for parsing SMS content using regular expressions	Python Standard Library
ctypes	Built-in	Used to display system-level pop-up messages via Windows API	Python Standard Library

7. Environment Setup Instructions

Installation

1. **Clone the repository:**

```
2. git clone https://github.com/Ahmed-Shahi/OS.git  
cd OS
```

3. **Install dependencies:**

```
pip install psutil pywin32
```

4. **Configure paths:** Update the Office paths in agent.py if different:

```
5. WORD_PATH = r"C:\Program Files\Microsoft Office\Office14\WINWORD.EXE"  
POWERPOINT_PATH = r"C:\Program Files\Microsoft Office\Office14\POWERPNT.EXE"
```

6. **Connect Android device:**

- o Enable USB debugging on your Android device
- o Connect via USB
- o Verify ADB connection: adb devices

Named Pipe

7. Communication between agent and service uses:

```
8. \\.\pipe\phone_control_pipe
```

Usage

1. **Start the agent:**

```
python agent.py
```

2. **Install and start the service:**

```
3. python service.py install  
python service.py start
```

4. **Send SMS commands:**

Open and close the office applications (Word, PPT) and shut down.

8. References

- [1] Python Software Foundation, *Python 3 Documentation*, Python Software Foundation, <https://docs.python.org/3/>, Accessed 2025.
- [2] Mark Hammond and Andy Robinson, *Python Programming on Win32*, O'Reilly Media, USA, 2000.
- [3] Microsoft Corporation, *Windows Services Overview*, Microsoft Documentation, <https://learn.microsoft.com/>, Accessed 2025.
- [4] Microsoft Corporation, *Named Pipes*, Microsoft Windows API Documentation, <https://learn.microsoft.com/>, Accessed 2025.
- [5] Giampaolo Rodola, *psutil: Cross-platform Library for Process and System Monitoring*, Python Package Documentation, <https://psutil.readthedocs.io/>, Accessed 2025.
- [6] Android Developers, *Android Debug Bridge (ADB)*, Google Developer Documentation, <https://developer.android.com/studio/command-line/adb>, Accessed 2025.
- [7] Python Software Foundation, *Python Standard Library Documentation*, Python Software Foundation, <https://docs.python.org/3/library/>, Accessed 2025.